FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 1

Name: Aufwand in h:

Mat.Nr: Punkte:

Übungsgruppe: korrigiert:

Beispiel 1 (24 Punkte) Hausverwaltung: Entwerfen Sie aus der nachfolgenden Spezifikation ein Klassendiagramm, instanzieren Sie dieses und implementieren Sie die Funktionalität entsprechend:

Zur Verwaltung von Häusern soll eine Software erstellt werden. Jedes Haus wird durch die Adresse eindeutig bestimmt und hat eine fixe Anzahl von Stockwerken und in diesen wieder eine fixe Anzahl von Wohnungen.

Eine Wohnung hat eine Türnummer, eine bestimmte Größe (m^2) und ev. einen Balkon.

Für jede Wohnung wird ein Hauptmieter eingetragen, von diesem müssen Vorname, Nachname und das Geburtsjahr angegeben werden.

Es muss möglich sein, alle Häuser mit Wohnungen und Mietern auszugeben (Print()- Methode).

Die Ausgabe kann zum Beispiel folgendermaßen aussehen:

```
House: Hasenstraße 45, 4040 Linz, 2 floors
  Floor 0
    Flat 1, 65qm, no balcony: Hans Huber (1950)
    Flat 2, 78qm, no balcony: Peter Bauer (1972)
  Floor 1
    Flat 3, 65qm, with balcony: Rudi Müller (1982)
    Flat 4, 78qm, with balcony: Klaus Hofer (1968)
House: Fuchsgasse 17, 4232 Hagenberg, 3 floors
  Floor 0
    Flat 1, 46qm, no balcony: Susi Huber (1960)
    Flat 2, 78qm, no balcony: Petra Neuhuber (1956)
  Floor 1
    Flat 3, 46qm, with balcony: Rudi Müller (1961)
    Flat 4, 78qm, with balcony: Clemens Mair (1977)
```

```
Floor 2
  Flat 5, 46qm, with balcony: Michael Bauer (1985)
  Flat 6, 78qm, no balcony: Maria Rieder (1962)
```

Überlegen Sie sich die jeweils notwendigen Members und Methoden der einzelnen Klassen. Schreiben Sie einen Testtreiber, der mehrere Häuser mit Wohnungen und Mietern anlegt und diese dann ausdruckt.

Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)! In dieser soll auch enthalten sein, wie Sie die Aufgabenstellung auf die Teamteilnehmer verteilt haben. Geben Sie auch in den entsprechenden Header-Dateien den Verfasser an!

Führen Sie gemeinsam vor der Realisierung eine Aufwandsschätzung in (Ph) durch und notieren Sie die geschätzte Zeitdauer in der Systemdokumentation. Dies gilt auch für alle nachfolgenden Übungen.

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!

HSD
OBERÖSTERREICH
FH-HAGENBERG

Systemdokumentation

PropertyManagement

Version 1.0

J. Rieger, L. Kieweg

Hagenberg, 16. Oktober 2025

Dieses Dokument zeigt den prinzipiellen Aufbau einer Systemdokumentation für Software-Projekte. Die einzelnen Kapitel sind mit Kommentaren versehen, welche die Struktur und den Inhalt erläutern.

Inhaltsverzeichnis

1 Organisatorisches **2**
1.1 Team . . . 2
1.2 Aufteilung der Verantwortungsbereiche . . . 2
1.3 Aufwand . . . 2

2 Anforderungsdefinition (Systemspezifikation) **2**

3 Systementwurf **3**
3.1 Klassendiagram . . . 3
3.2 Designentscheidungen . . . 3

4 Komponentenentwurf **3**

5 Testprotokollierung **4**

6 Quellcode **4**

1 Organisatorisches

1.1 Team

- Julian Rieger, Matr.-Nr: S2510306023, E-Mail: S25010306023@fhooe.at
- Leander Kieweg, Matr.-Nr: S2510306014, E-Mail: S2510306014@fhooe.at

1.2 Aufteilung der Verantwortungsbereiche

- Julian Rieger
  - Implementierung der Klassen: PropertyManagement, House
  - Implementierung des Testtreibers
  - Design Klassendiagram
  - Verfassen der Systemdokumentation
- Leander Kieweg
  - Implementierung der Klassen: Floor, Flat, Tenant
  - Design des Klassendiagrams
  - Überarbeitung und generelle Code-Optimierung

1.3 Aufwand

- Julian Rieger: geschätzt: 5h, tatsächlich 7h
- Leander Kieweg: geschätzt 5h, tatsächlich 7h

2 Anforderungsdefinition (Systemspezifikation)

In diesem System wird eine Verwaltungssoftware für eine Hausverwaltung bereitsgestellt. Das System erfolgt durch eine verschachtelte Klassenstruktur bestehend aus Mieter (Tenant), Wohnung (Flat), Etage (Floor), Wohnhaus (House) und die Hausverwaltung (PropertyManagement). Die Hausverwaltung liefert folgende Funktionalität:

- Das Hinzufügen eines neuen Wohnhauses
- Das Entfernen eines bereits registrierten Wohnhauses
- Das Ausgeben der registrierten Daten aller Häuser der Hausverwaltung.

Nach dem erstellen eines Wohnhauses ist die Veränderung dieses Objekts nicht mehr möglich. Es ist lediglich der Vermieter frei veränderbar, falls Wohnung oder Etagen hinzukommen oder entfernt werden ist es notwendig, ein neues Haus dafür zu registrieren und das alte Objekt mit der gegebenen Methode zu entfernen.

3 Systementwurf

3.1 Klassendiagram

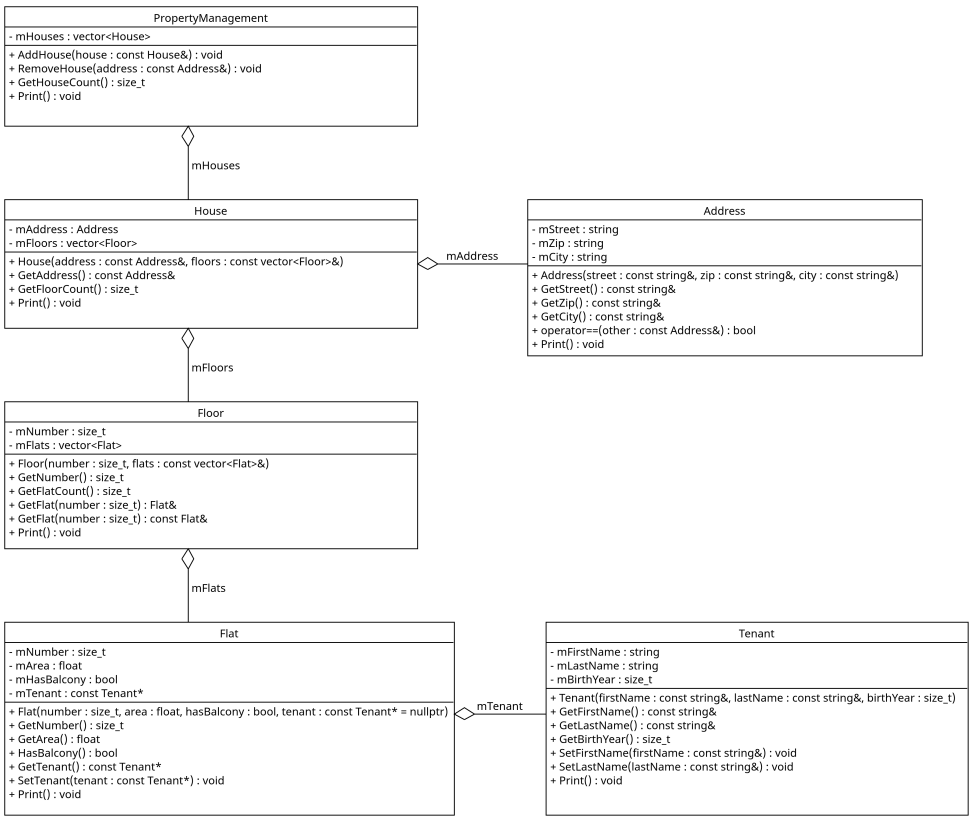


Abbildung 1: Klassendiagram PropertyManagement

3.2 Designentscheidungen

Als Design wurde eine Verschachtelung der Klassen gewählt, da eine has-A-Beziehung besteht. Für die Speicherung der Objekte wird ein Vektor verwendet, um die Verwaltung der Objekte übersichtlich und dynamisch veränderbar zu gestalten. Zuerst wurde die Verwendung einer Hashmap in Betracht gezogen, jedoch entschieden wir uns bewusst dagegen. Dafür spricht, die Assoziation mit Addressen eindeutig zu halten, jedoch fiel die Wahl schlussendlich auf den Vektor, da wir kein Objekt als Schlüssel verwenden wollten, und um Redundanzen zu vermeiden, da sonst die Addresse zwei Mal gespeichert werden würde. Wir haben auch eine SetTenant-Methode implementiert, die nicht aus der Angabe hervorgeht, wir jedoch als essenziell erachtet haben, um eine brauchbare Verwaltung der Wohnungen zu ermöglichen.

4 Komponentenentwurf

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

```
House: Softwarepark 47, 4232 Hagenberg, 2 floors
  Floor 0
    Flat 1, 45qm, no balcony: Hubert von Goisern (1960)
    Flat 2, 90qm, with balcony: Jakob Schubert (1990)
  Floor 1
    Flat 3, 63.5qm, no balcony: Ayrton Senna (1950)
    Flat 4, 105qm, with balcony: vacant

House: Musterstrasse 14, 4711 Linz, 2 floors
  Floor 0
    Flat 1, 39qm, with balcony: Maximilian Laimer (1960)
    Flat 2, 65qm, no balcony: Leander Mueller (2000)
  Floor 1
    Flat 3, 70qm, with balcony: Manuel Hundstorfer (1995)
    Flat 4, 70qm, with balcony: Manuel Hundstorfer (1995)


[SUCCESS] Address::GetCity()
[SUCCESS] Address::GetStreet()
[SUCCESS] Address::GetZip()
[SUCCESS] House::GetFloorCount()
[SUCCESS] Floor::GetNumber()
[SUCCESS] Floor::GetFlatCount()
[SUCCESS] Floor::GetFlat(1).GetNumber()
[SUCCESS] Flat::GetArea()
[SUCCESS] Flat::HasBalcony()
[SUCCESS] Flat::HasBalcony()
[SUCCESS] Flat::GetTenant() != nullptr
[SUCCESS] Tenant::GetBirthYear()
[SUCCESS] Tenant::GetFirstName()
[SUCCESS] Tenant::GetLastName()
[SUCCESS] Floor::GetFlat() const overload
[SUCCESS] Flat::SetTenant(newTenant)
[SUCCESS] Flat::SetTenant(nullptr)
[SUCCESS] Flat::SetTenant(restore)
[SUCCESS] Address throws invalid_argument on empty street
[SUCCESS] House throws invalid_argument on empty floors
[SUCCESS] Floor throws invalid_argument on empty flats
[SUCCESS] Floor::GetFlat throws out_of_range for missing door number
[SUCCESS] PropertyManagement::AddHouse throws on duplicate address
[SUCCESS] PropertyManagement count before removal == 2
[SUCCESS] PropertyManagement::RemoveHouse reduces count to 1
[SUCCESS] PropertyManagement::RemoveHouse throws on unknown address
```

6 Quellcode

Listing 1: `Tenant.h`

```h
#ifndef TENANT_H
#define TENANT_H

#include <cstddef>
#include <string>

/// \file Tenant.h
/// \author Leander Kieweg

/// \brief A main tenant of a flat.
class Tenant {
public:
    /// \brief Creates a tenant.
    /// \param firstName First name.
    /// \param lastName Last name.
    /// \param birthYear Year of birth.
    Tenant(const std::string& firstName, const std::string& lastName, size_t
        birthYear);

    /// \brief Gets the first name.
    /// \return The first name.
    const std::string& GetFirstName() const;

    /// \brief Gets the last name.
    /// \return The last name.
    const std::string& GetLastName() const;

    /// \brief Changes the first name.
    /// \param firstName The new first name.
    void SetFirstName(const std::string& firstName);

    /// \brief Changes the last name, for example after a marriage.
    /// \param lastName The new last name.
    void SetLastName(const std::string& lastName);

    /// \brief Gets the year of birth.
    /// \return The year of birth.
    size_t GetBirthYear() const;

    /// \brief Prints the tenant to the console, for example: Hans Huber (1950).
    ///        No line break at the end.
    void Print() const;

private:
    std::string mFirstName;
    std::string mLastName;
    size_t mBirthYear;
};

#endif
```

Listing 2: `Tenant.cpp`

```cpp
/// \file Tenant.cpp
/// \author Leander Kieweg

#include "Tenant.h"
#include <iostream>

Tenant::Tenant(const std::string& firstName, const std::string& lastName, size_t
    birthYear)
    : mFirstName(firstName), mLastName(lastName), mBirthYear(birthYear) {}

const std::string& Tenant::GetFirstName() const { return mFirstName; }

const std::string& Tenant::GetLastName() const { return mLastName; }

```

```cpp
void Tenant::SetFirstName(const std::string& firstName) { mFirstName = firstName; }

void Tenant::SetLastName(const std::string& lastName) { mLastName = lastName; }

size_t Tenant::GetBirthYear() const { return mBirthYear; }

void Tenant::Print() const {
    std::cout << mFirstName << " " << mLastName << " (" << mBirthYear << ")";
}
```

Listing 3: `Flat.h`

```cpp
#ifndef FLAT_H
#define FLAT_H

#include "Tenant.h"

/// \file Flat.h
/// \author Leander Kieweg

/// \brief A flat. It can have a main tenant or be empty.
///
/// The flat does not own the tenant. It only points to it.
/// The caller must keep the tenant alive as long as the flat uses it.
class Flat {
public:
    /// \brief Creates a flat.
    /// \param number Door number.
    /// \param area Size in square meters.
    /// \param hasBalcony True if the flat has a balcony.
    /// \param tenant Main tenant. nullptr means the flat is empty.
    Flat(size_t number, float area, bool hasBalcony, const Tenant* tenant =
        nullptr);

    /// \brief Gets the door number.
    /// \return The door number.
    size_t GetNumber() const;

    /// \brief Gets the size.
    /// \return The size in square meters.
    float GetArea() const;

    /// \brief Tells if the flat has a balcony.
    /// \return True if there is a balcony.
    bool HasBalcony() const;

    /// \brief Gets the main tenant.
    /// \return Pointer to the tenant, or nullptr if the flat is empty.
    const Tenant* GetTenant() const;

    /// \brief Changes the main tenant.
    /// \param tenant The new tenant. nullptr makes the flat empty.
    void SetTenant(const Tenant* tenant);

    /// \brief Prints the flat as one line with 4 spaces indent.
    ///        An empty flat shows "vacant" instead of a tenant.
    void Print() const;

private:
    size_t mNumber;
    float mArea;
    bool mHasBalcony;
    const Tenant* mTenant;
};

#endif
```

Listing 4: `Flat.cpp`

```cpp
/// \file Flat.cpp
/// \author Leander Kieweg

#include "Flat.h"
#include <iostream>

Flat::Flat(size_t number, float area, bool hasBalcony, const Tenant* tenant)
    : mNumber(number), mArea(area), mHasBalcony(hasBalcony), mTenant(tenant) {}

size_t Flat::GetNumber() const { return mNumber; }

float Flat::GetArea() const { return mArea; }

bool Flat::HasBalcony() const { return mHasBalcony; }

const Tenant* Flat::GetTenant() const { return mTenant; }

void Flat::SetTenant(const Tenant* tenant) { mTenant = tenant; }

void Flat::Print() const {
    std::cout << "    Flat " << mNumber << ", " << mArea << "qm, "
              << (mHasBalcony ? "with balcony" : "no balcony") << ": ";
    if (mTenant != nullptr) {
        mTenant->Print();
    } else {
        std::cout << "vacant";
    }
}
```

Listing 5: `Floor.h`

```cpp
#ifndef FLOOR_H
#define FLOOR_H

#include "Flat.h"
#include <vector>

/// \file Floor.h
/// \author Leander Kieweg

/// \brief A floor of a house with a fixed number of flats.
///
/// The flats are given at creation. Flats cannot be added or removed later.
class Floor {
public:
    /// \brief Creates a floor.
    /// \param number Floor number. The ground floor is 0.
    /// \param flats The flats on this floor. Must not be empty.
    /// \throws std::invalid_argument If flats is empty.
    Floor(size_t number, const std::vector<Flat>& flats);

    /// \brief Gets the floor number.
    /// \return The floor number.
    size_t GetNumber() const;

    /// \brief Gets the number of flats.
    /// \return The number of flats on this floor.
    size_t GetFlatCount() const;

    /// \brief Gets a flat by its door number so its tenant can be changed.
    /// \param number Door number of the flat.
    /// \return Reference to the flat.
    /// \throws std::out_of_range If no flat with this number exists.
    Flat& GetFlat(size_t number);

    /// \brief Gets a flat by its door number for read-only access.
    /// \param number Door number of the flat.
    /// \return Const reference to the flat.
```

```
38      /// \throws std::out_of_range If no flat with this number exists.
39      const Flat& GetFlat(size_t number) const;
40
41      /// \brief Prints the floor with 2 spaces indent, followed by its flats.
42      void Print() const;
43
44  private:
45      size_t mNumber;
46      std::vector<Flat> mFlats;
47  };
48
49  #endif
```

Listing 6: `Floor.cpp`

```
1   /// \file Floor.cpp
2   /// \author Leander Kieweg
3
4   #include "Floor.h"
5   #include <iostream>
6   #include <stdexcept>
7   #include <algorithm>
8
9   Floor::Floor(size_t number, const std::vector<Flat>& flats)
10      : mNumber(number), mFlats(flats) {
11      if (mFlats.empty()) {
12          throw std::invalid_argument("Floor needs at least one flat");
13      }
14  }
15
16  size_t Floor::GetNumber() const { return mNumber; }
17
18  size_t Floor::GetFlatCount() const { return mFlats.size(); }
19
20  Flat& Floor::GetFlat(size_t number) {
21      auto it = std::find_if(mFlats.begin(), mFlats.end(),
22          [number](const Flat& f) { return f.GetNumber() == number; });
23      if (it == mFlats.end()) {
24          throw std::out_of_range("Flat not found");
25      }
26      return *it;
27  }
28
29  const Flat& Floor::GetFlat(size_t number) const {
30      auto it = std::find_if(mFlats.cbegin(), mFlats.cend(),
31          [number](const Flat& f) { return f.GetNumber() == number; });
32      if (it == mFlats.cend()) {
33          throw std::out_of_range("Flat not found");
34      }
35      return *it;
36  }
37
38  void Floor::Print() const {
39      std::cout << "  Floor " << mNumber << "\n";
40      for (const Flat& flat : mFlats) {
41          flat.Print();
42          std::cout << std::endl;
43      }
44  }
```

Listing 7: `Address.h`

```
1   #ifndef ADDRESS_H
2   #define ADDRESS_H
3
4   #include <string>
5
```

```
6  /// \file Address.h
7  /// \author Julian Rieger
8
9  /// \brief Address contains of Street Name and Zipcode
10 ///
11 /// Zipcode and Address are saved as std::string
12 class Address
13 {
14 public:
15     /// \brief Creates address.
16     /// \param zip The Zipcode of the address
17     /// \param street The sreet of the address
18     /// \throws std::invalid_argument if either argument is empty.
19     Address(const std::string& street, const std::string& zip, const std::string&
           city);
20
21     /// \brief Compares two addresses for equality.
22     /// \param other The address to compare with.
23     /// \return True if street, zip, and city match.
24     bool operator==(const Address& other) const;
25
26     /// \brief Returns member mZip.
27     std::string GetZip() const;
28
29     /// \brief Returns member mStreet.
30     std::string GetStreet() const;
31
32     /// \brief Returns member mCity.
33     std::string GetCity() const;
34
35     /// \brief Prints Address formated on console.
36     void Print() const;
37 private:
38     std::string mZip;
39     std::string mStreet;
40     std::string mCity;
41 };
42
43 #endif
```

Listing 8: `Address.cpp`

```
1  /// \file Address.cpp
2  /// \author Julian Rieger
3
4  #include "Address.h"
5  #include <stdexcept>
6  #include <iostream>
7
8  Address::Address(const std::string& street, const std::string& zip, const std::
       string& city) :
9      mZip(zip),
10     mStreet(street),
11     mCity(city)
12 {
13     if (zip.empty() || street.empty() || city.empty())
14     {
15         throw std::invalid_argument("Invalid argument: Zip code, city or street
               name is empty.");
16     }
17 }
18
19 bool Address::operator==(const Address& other) const
20 {
21     return mStreet == other.mStreet && mZip == other.mZip && mCity == other.mCity;
22 }
23
24 std::string Address::GetZip() const
25 {
```

```
26      return mZip;
27  }
28
29  std::string Address::GetStreet() const
30  {
31      return mStreet;
32  }
33
34  std::string Address::GetCity() const
35  {
36      return mCity;
37  }
38
39  void Address::Print() const
40  {
41      std::cout << mStreet << ", " << mZip << " " << mCity;
42  }
```

Listing 9: House.h

```
1   #ifndef HOUSE_H
2   #define HOUSE_H
3
4   /// \file House.h
5   /// \author Julian Rieger
6
7   #include "Floor.h"
8   #include "Address.h"
9
10  /// \brief Represents a house uniquely identified by an address with a fixed set
        of floors.
11  class House
12  {
13  public:
14      /// \brief Creates a house.
15      /// \param address The postal address of the house.
16      /// \param floors Vector containing the floors of this house. Must not be
            empty.
17      /// \throws std::invalid_argument If the floors vector is empty.
18      House(const Address& address, const std::vector<Floor>& floors);
19
20      /// \brief Prints the house and all its floors and flats to std::cout.
21      void Print() const;
22
23      /// \brief Gets the address of the house.
24      /// \return Const reference to the address.
25      const Address& GetAddress() const;
26
27      /// \brief Gets the total number of floors.
28      /// \return The number of floors in the house.
29      size_t GetFloorCount() const;
30
31  private:
32      std::vector<Floor> mFloors;
33      Address mAddress;
34  };
35
36  #endif
```

Listing 10: House.cpp

```
1   /// \file House.cpp
2   /// \author Julian Rieger
3
4   #include "House.h"
5   #include <stdexcept>
6   #include <iostream>
```

```
7
8  House::House(const Address &address, const std::vector<Floor>& floors) :
9      mAddress(address),
10     mFloors(floors)
11 {
12     if (mFloors.empty())
13     {
14         throw std::invalid_argument("Invalid vector: Cannot add empty floor vector
               to object house.");
15     }
16 }
17
18 const Address& House::GetAddress() const
19 {
20     return mAddress;
21 }
22
23 size_t House::GetFloorCount() const
24 {
25     return mFloors.size();
26 }
27
28 void House::Print() const
29 {
30     std::cout << "House: ";
31     mAddress.Print();
32     std::cout << ", " << mFloors.size() << " floors\n";
33     for (const auto& floor : mFloors)
34     {
35         floor.Print();
36     }
37 }
```

Listing 11: PropertyManagement.h

```
1  #ifndef PROPERTYMANAGEMENT_H
2  #define PROPERTYMANAGEMENT_H
3
4  #include "House.h"
5  #include <vector>
6
7  /// \file PropertyManagement.h
8  /// \author Julian Rieger
9
10 /// \brief Object containing existing houses and managing them.
11 class PropertyManagement
12 {
13 public:
14     /// \brief Prints all houses.
15     void Print() const;
16
17     /// \brief Adds a house if its address is unique.
18     /// \param house The house to add.
19     /// \throws std::invalid_argument If a house with this address already exists.
20     void AddHouse(const House& house);
21
22     /// \brief Removes a house by its address.
23     /// \param address Address of the house to remove.
24     /// \throws std::out_of_range If no house with this address is found.
25     void RemoveHouse(const Address& address);
26
27     /// \brief Returns the number of managed houses.
28     /// \return Number of houses.
29     size_t GetHouseCount() const;
30
31 private:
32     std::vector<House> mHouses;
33 };
34
```

```
35 #endif
```

Listing 12: PropertyManagement.cpp

```
1  /// \file PropertyManagement.cpp
2  /// \author Julian Rieger
3
4  #include "PropertyManagement.h"
5  #include <iostream>
6  #include <algorithm>
7  #include <stdexcept>
8
9  void PropertyManagement::Print() const
10 {
11     for (const auto& house : mHouses)
12     {
13         house.Print();
14         std::cout << "\n";
15     }
16 }
17
18 void PropertyManagement::AddHouse(const House& house)
19 {
20     auto it = std::find_if(mHouses.cbegin(), mHouses.cend(),
21         [&house](const House& h) {
22             return h.GetAddress() == house.GetAddress();
23         });
24
25     if (it != mHouses.cend())
26     {
27         throw std::invalid_argument("House with this address already exists.");
28     }
29
30     mHouses.push_back(house);
31 }
32
33 void PropertyManagement::RemoveHouse(const Address& address)
34 {
35     auto it = std::find_if(mHouses.begin(), mHouses.end(),
36         [&address](const House& h) {
37             return h.GetAddress() == address;
38         });
39
40     if (it == mHouses.end())
41     {
42         throw std::out_of_range("House with this address not found.");
43     }
44
45     mHouses.erase(it);
46 }
47
48 size_t PropertyManagement::GetHouseCount() const
49 {
50     return mHouses.size();
51 }
```

Listing 13: main.cpp

```
1  /// \file main.cpp
2  /// \brief Test driver. Runs all tests and prints the results.
3  /// \author Leander Kieweg, Julian Rieger
4
5  #include "PropertyManagement.h"
6  #include <iostream>
7  #include <string>
8
9  /// \brief Helper function to report test outcomes.
```

```
10 void CheckTest(bool condition, const std::string& testName)
11 {
12     if (condition)
13     {
14         std::cout << "[SUCCESS] " << testName << std::endl;
15     }
16     else
17     {
18         std::cerr << "[FAILED]  " << testName << std::endl;
19     }
20 }
21
22 void RunTestDriver();
23
24 /// \brief Starts the program.
25 /// \return 0.
26 int main()
27 {
28     RunTestDriver();
29     return 0;
30 }
31
32 void RunTestDriver()
33 {
34     // Test data setup
35     Tenant t1{ "Hubert", "von Goisern", 1960 };
36     Tenant t2{ "Jakob", "Schubert", 1990 };
37     Tenant t3{ "Ayrton", "Senna", 1950 };
38     Tenant t4{ "Maximilian", "Laimer", 1960 };
39     Tenant t5{ "Leander", "Mueller", 2000 };
40     Tenant t6{ "Manuel", "Hundstorfer", 1995 };
41     Tenant t7{ "Daniel", "Moser", 1949 };
42
43     Flat f1{ 1, 45.0f, false, &t1 };
44     Flat f2{ 2, 90.0f, true, &t2 };
45     Flat f3{ 3, 63.5f, false, &t3 };
46     Flat f4{ 4, 105.0f, true }; // Empty flat
47     Flat f5{ 1, 39.0f, true, &t4 };
48     Flat f6{ 2, 65.0f, false, &t5 };
49     Flat f7{ 3, 70.0f, true, &t6 };
50     Flat f8{ 4, 70.0f, true, &t6 };
51
52     std::vector<Flat> floor1{ f1, f2 };
53     std::vector<Flat> floor2{ f3, f4 };
54     std::vector<Flat> floor3{ f5, f6 };
55     std::vector<Flat> floor4{ f7, f8 };
56
57     Floor e1{ 0, floor1 };
58     Floor e2{ 1, floor2 };
59     Floor e3{ 0, floor3 };
60     Floor e4{ 1, floor4 };
61
62     std::vector<Floor> house1{ e1, e2 };
63     std::vector<Floor> house2{ e3, e4 };
64
65     House h1{ {"Softwarepark 47", "4232", "Hagenberg"}, house1 };
66     House h2{ {"Musterstrasse 14", "4711", "Linz"}, house2 };
67
68     PropertyManagement prop;
69     prop.AddHouse(h1);
70     prop.AddHouse(h2);
71
72     // Demonstration output
73     prop.Print();
74     std::cout << std::endl;
75
76     // Getter tests
77     CheckTest(h1.GetAddress().GetCity() == "Hagenberg", "Address::GetCity()");
78     CheckTest(h1.GetAddress().GetStreet() == "Softwarepark 47", "Address::
           GetStreet()");
```

```
79      CheckTest(h1.GetAddress().GetZip() == "4232", "Address::GetZip()");
80      CheckTest(h1.GetFloorCount() == 2, "House::GetFloorCount()");
81
82      CheckTest(e1.GetNumber() == 0, "Floor::GetNumber()");
83      CheckTest(e1.GetFlatCount() == 2, "Floor::GetFlatCount()");
84
85      CheckTest(e1.GetFlat(1).GetNumber() == 1, "Floor::GetFlat(1).GetNumber()");
86      CheckTest(e1.GetFlat(1).GetArea() == 45.0f, "Flat::GetArea()");
87      CheckTest(e1.GetFlat(1).HasBalcony() == false, "Flat::HasBalcony()");
88      CheckTest(e1.GetFlat(2).HasBalcony() == true, "Flat::HasBalcony()");
89
90      CheckTest(e1.GetFlat(1).GetTenant() != nullptr, "Flat::GetTenant() != nullptr"
            );
91      CheckTest(e1.GetFlat(1).GetTenant()->GetBirthYear() == 1960, "Tenant::
            GetBirthYear()");
92      CheckTest(e1.GetFlat(1).GetTenant()->GetFirstName() == "Hubert", "Tenant::
            GetFirstName()");
93      CheckTest(e1.GetFlat(1).GetTenant()->GetLastName() == "von Goisern", "Tenant::
            GetLastName()");
94
95      const Floor& constFloorRef = e1;
96      CheckTest(constFloorRef.GetFlat(1).GetNumber() == 1, "Floor::GetFlat() const
            overload");
97
98      // Changing tenant
99      e1.GetFlat(1).SetTenant(&t7);
100     CheckTest(e1.GetFlat(1).GetTenant() == &t7, "Flat::SetTenant(newTenant)");
101
102     e1.GetFlat(1).SetTenant(nullptr);
103     CheckTest(e1.GetFlat(1).GetTenant() == nullptr, "Flat::SetTenant(nullptr)");
104
105     e1.GetFlat(1).SetTenant(&t1);
106     CheckTest(e1.GetFlat(1).GetTenant() == &t1, "Flat::SetTenant(restore)");
107
108     // Exception tests
109     try
110     {
111         House invalidHouse{ {"", "4711", "Hagenberg"}, house1 };
112         CheckTest(false, "Address throws invalid_argument on empty street");
113     }
114     catch (const std::invalid_argument&)
115     {
116         CheckTest(true, "Address throws invalid_argument on empty street");
117     }
118
119     try
120     {
121         std::vector<Floor> emptyFloors{};
122         House invalidHouse{ {"Softwarepark 11", "4711", "Hagenberg"}, emptyFloors
                };
123         CheckTest(false, "House throws invalid_argument on empty floors");
124     }
125     catch (const std::invalid_argument&)
126     {
127         CheckTest(true, "House throws invalid_argument on empty floors");
128     }
129
130     try
131     {
132         std::vector<Flat> emptyFlats{};
133         Floor invalidFloor{ 0, emptyFlats };
134         CheckTest(false, "Floor throws invalid_argument on empty flats");
135     }
136     catch (const std::invalid_argument&)
137     {
138         CheckTest(true, "Floor throws invalid_argument on empty flats");
139     }
140
141     try
142     {
```

```
143         e1.GetFlat(999);
144         CheckTest(false, "Floor::GetFlat throws out_of_range for missing door
                number");
145     }
146     catch (const std::out_of_range&)
147     {
148         CheckTest(true, "Floor::GetFlat throws out_of_range for missing door
                number");
149     }
150
151     try
152     {
153         prop.AddHouse(h1);
154         CheckTest(false, "PropertyManagement::AddHouse throws on duplicate address
                ");
155     }
156     catch (const std::invalid_argument&)
157     {
158         CheckTest(true, "PropertyManagement::AddHouse throws on duplicate address"
                );
159     }
160
161     CheckTest(prop.GetHouseCount() == 2, "PropertyManagement count before removal
            == 2");
162     prop.RemoveHouse(h1.GetAddress());
163     CheckTest(prop.GetHouseCount() == 1, "PropertyManagement::RemoveHouse reduces
            count to 1");
164
165     try
166     {
167         prop.RemoveHouse(h1.GetAddress());
168         CheckTest(false, "PropertyManagement::RemoveHouse throws on unknown
                address");
169     }
170     catch (const std::out_of_range&)
171     {
172         CheckTest(true, "PropertyManagement::RemoveHouse throws on unknown address
                ");
173     }
174 }
```